

When Flow meet Reinforcement Learning

Hua Hsuan Liang, Jaisel Singh, and Edward Zhang
Columbia University

Abstract—Flow and diffusion-based policies have recently achieved strong performance in imitation learning (IL), but IL alone cannot generalize reliably when deployed in real environments due to distribution shift and the lack of interaction. We therefore ask *when* embedding Continuous Flow Matching (CFM) into reinforcement learning (RL) actually helps: can expressive flow policies improve exploration without destroying credit assignment over long horizons? We propose two approaches: **Flow Policy Optimization (FPO)**, an on-policy method that optimizes denoising trajectories with policy gradients, and **Flow Soft Actor-Critic (FSAC)**, an off-policy variant that embeds flow generation into value learning. To support FSAC, we introduce a *Dual-MDP* formulation that models the denoising process as a short-horizon auxiliary MDP nested inside the environment MDP. Experiments on continuous-control benchmarks show that flow-guided action generation can improve local exploration but also induces a *horizon compounding* effect: expanding each environment step into many flow steps weakens value propagation and degrades performance on complex tasks. Our results therefore highlight both the promise and the structural limitations of bringing flow-based generative policies from IL into RL.

I. INTRODUCTION

Imitation learning (IL) has emerged as a powerful paradigm for robotic control, particularly with the recent success of diffusion- and flow-based generative policies. Prior work has demonstrated that denoising- or flow-based models can capture complex, multimodal expert behavior and achieve strong performance across manipulation benchmarks [1, 2, 3]. Continuous Flow Matching (CFM) [4, 5, 6] provides a principled framework for learning generative trajectories, offering smoother action distributions and richer expressivity compared to traditional Gaussian policies widely used in reinforcement learning (RL) [7, 8].

Despite these advantages, IL fundamentally suffers from distribution shift: policies trained purely from demonstrations cannot recover when encountering unseen states during deployment [9]. Without the ability to interact with the environment, imitation-only policies accumulate compounding errors and fail in long-horizon, real-world robotic tasks. Even advanced generative models such as diffusion or flow-based IL [1, 3] inherit this limitation.

RL, in contrast, enables continual policy improvement through trial-and-error interaction. Integrating flow-based generative policies into RL is therefore a promising direction: these expressive models could retain their multimodal action-generation capabilities while gaining robustness through adaptive learning. However, doing so introduces challenges. The multi-step flow-generation process adds temporal structure that complicates credit assignment, interacts non-trivially with the environment dynamics, and is incompatible with standard RL

updates that assume single-step action sampling [10, 11].

To address these challenges, we study how Continuous Flow Matching (CFM) can be embedded into both on-policy SAC [8] and off-policy PPO [7] RL algorithms. We propose two complementary approaches. First, **Flow Policy Optimization (FPO)** is an on-policy method that directly optimizes flow trajectories using policy gradients, building conceptually on flow-based policy optimization ideas [12]. Second, **Flow Soft Actor-Critic (FSAC)** incorporates flow-based action generation into an off-policy value-learning framework [8]. To support FSAC, we introduce a novel *Dual-MDP* formulation that treats the flow-matching denoising process as a short-horizon auxiliary MDP coupled to the underlying environment dynamics.

To eventually support IL→RL fine-tuning, our broader aim is to use pretrained flow-based imitation policies as strong action priors and refine them through online reinforcement learning. As a first step, however, we focus on validating our proposed algorithms in a controlled setting by training them from scratch, ensuring that the core flow-based RL mechanisms function correctly before incorporating pretrained IL models.

A. Contributions

Our main contributions are summarized as follows:

- **Unified formulation for flow-based RL:** We formalize how Continuous Flow Matching (CFM) can be integrated into reinforcement learning and characterize the algorithmic challenges that arise when generative flow processes interact with environment dynamics.
- **Flow Policy Optimization (FPO):** An on-policy RL method that optimizes full flow trajectories using policy gradients, using recent advances in flow-based policy optimization [12].
- **Flow Soft Actor-Critic (FSAC):** An off-policy RL algorithm that embeds flow-based action generation into value learning, extending the maximum-entropy RL framework [8].
- **Dual-MDP framework:** A novel interpretation of the CFM denoising process as a short-horizon auxiliary MDP, which is inspired by DPPO [13], enabling Q-learning to operate over flow states and intermediate latent trajectories.

II. RELATED WORK

A. Imitation Learning and Demonstration-Driven Policies

Imitation learning (IL) remains a widely used strategy for robotic control due to its simplicity and sample efficiency.

Classical methods such as behavior cloning and DAGGER [9] address distribution shift via dataset aggregation, while adversarial imitation methods such as GAIL [14] aim to recover expert-like behavior through discriminator feedback.

More recently, generative IL models have become dominant in visuomotor control, including diffusion-based visuomotor policies [1], planning-based diffusion models for long-horizon behavior [2], and multimodal manipulation policies using flow-matching [3]. These models learn richer and smoother action distributions, but they still inherit the core limitation of IL: they cannot adapt to unseen states without interaction [15, 9].

B. Diffusion and Flow-Matching Generative Models

Diffusion models have demonstrated strong generative capabilities across images, trajectories, and robotic actions [16, 2]. Flow Matching (FM) and continuous-time vector field modeling [4, 5, 6] further improve generative efficiency by learning continuous flows without requiring noise schedules.

Extensions to flow-based policy learning include Flow Matching Policy Gradients (FMPG) [12], reward-weighted flow fine-tuning [17], and Variational Flow Policies [3]. Diffusion-based RL research has also explored optimizing denoising trajectories directly, as in DPPO [13]. These works provide inspiration for our on-policy FPO and off-policy FSAC designs.

C. Reinforcement Learning for Continuous Control

Classical RL algorithms such as PPO [7], SAC [8], and TD3 [10], along with educational frameworks such as SpinningUp [11], have demonstrated strong performance in continuous control. However, these methods rely on simple unimodal Gaussian policies that struggle with multimodal action distributions or ambiguous tasks.

Recent work has examined expressive policy classes such as normalizing flows [8, 18] and energy-based policies [19]. Yet, integrating multi-step generative processes (e.g., diffusion, flow matching) into RL remains challenging due to temporal abstraction, credit assignment, and compatibility with off-policy training.

D. Integrating Generative Models with Reinforcement Learning

There is growing interest in combining generative trajectory models with RL to improve robustness, exploration, and generalization. Methods such as DPPO [13], FPO [12], and online reward-weighted flow learning [17] demonstrate that generative denoising processes can be optimized with RL objectives.

However, nearly all prior work focuses on **on-policy** optimization or supervised finetuning. To the best of our knowledge, no prior method integrates continuous flow-matching policies into an **off-policy** RL framework with replay buffers and Q-function learning. Our Flow Soft Actor-Critic (FSAC) builds upon inspiration from DPPO [13] using *Dual-MDP* formulation that allows Q-functions to reason about intermediate flow states, enabling stable off-policy training.

Overall, our work unifies insights from imitation learning, diffusion and flow generative models, and reinforcement learning to develop a principled framework for flow-based policy optimization in both on-policy and off-policy settings.

III. METHODOLOGY

A. Preliminaries: PPO and SAC

We briefly review the PPO and SAC objectives, which serve as the foundation for our Flow Policy Optimization (FPO) and Flow Soft Actor-Critic (FSAC) methods.

a) *Proximal Policy Optimization (PPO)*: PPO maximizes a clipped policy-gradient objective [7]:

$$\mathcal{L}_{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(r_\theta A_t, \text{clip}(r_\theta, 1 - \epsilon, 1 + \epsilon) A_t \right) \right], \quad (1)$$

where the likelihood ratio is

$$r_\theta = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (2)$$

b) *Soft Actor-Critic (SAC)*: SAC maximizes an entropy-regularized policy objective [8]:

$$J_{\text{SAC}}(\theta) = \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi_\theta} \left[\alpha \mathcal{H}(\pi_\theta(\cdot | s_t)) - Q_\phi(s_t, a_t) \right]. \quad (3)$$

The critic minimizes the soft Bellman residual:

$$J_Q(\phi) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{D}} \left[(Q_\phi(s_t, a_t) - y_t)^2 \right], \quad (4)$$

where the target value is

$$y_t = r_t + \gamma \mathbb{E}_{a_{t+1} \sim \pi_\theta} \left[Q_\phi(s_{t+1}, a_{t+1}) - \alpha \log \pi_\theta(a_{t+1} | s_{t+1}) \right]. \quad (5)$$

B. Flow Policy Optimization (FPO)

Flow Policy Optimization (FPO) adapts PPO to *flow-based generative policies*, where actions are generated by integrating a learned velocity field rather than sampling from an explicit density. As a result, the exact policy likelihood $\log \pi_\theta(a_t | o_t)$ is generally *intractable*.

a) *Flow Matching as a Surrogate Objective.*: Instead of maximizing log-likelihood directly, flow-based policies are trained via a *flow matching loss*, which encourages the learned velocity field to match a target vector field along a continuous-time trajectory. For a given action sample a_t , this loss takes the form

$$\mathcal{L}_{\text{FM}}(\theta; a_t, o_t) = \mathbb{E}_\tau \left[\|v_\theta(a_\tau, \tau | o_t) - v^*(a_\tau, \tau)\|^2 \right], \quad (6)$$

where $\tau \in [0, 1]$ denotes the flow time and v^* is the target vector field.

b) *Connection to ELBO.*: As shown in [12], minimizing the flow matching loss is equivalent (up to constants) to maximizing a tractable *Evidence Lower Bound (ELBO)* on the intractable log-likelihood:

$$\text{ELBO}_\theta(a_t | o_t) = \log \pi_\theta(a_t | o_t) - \mathcal{D}_\theta^{\text{KL}}, \quad (7)$$

$$\mathcal{L}_\theta^w(a_t) = \mathcal{L}_{\text{FM}}(\theta; a_t, o_t) = -\text{ELBO}_\theta(a_t | o_t) + c, \quad (8)$$

where $\mathcal{D}_\theta^{\text{KL}}$ denotes the KL divergence between the learned flow-induced distribution and the ideal target distribution.

Although this KL term cannot be computed explicitly, it is implicitly minimized through flow matching, where c is a constant independent of θ . In practice, this corresponds to weighting the flow matching loss by PPO-style advantages while maintaining stability through clipping.

c) *Likelihood Ratio via ELBO*.: PPO relies on the likelihood ratio $\pi_\theta/\pi_{\theta_{\text{old}}}$, which is unavailable in closed form for flow policies. FPO therefore replaces it with an *ELBO-based ratio*:

$$r^{\text{FPO}}(\theta) = \frac{\exp(\text{ELBO}_\theta(a_t | o_t))}{\exp(\text{ELBO}_{\theta_{\text{old}}}(a_t | o_t))}. \quad (9)$$

d) *Factorization Into Likelihood and KL Gap*.: Substituting Eq. 7 yields

$$r^{\text{FPO}}(\theta) = \frac{\pi_\theta(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)} \cdot \frac{\exp(\mathcal{D}_{\theta_{\text{old}}}^{\text{KL}})}{\exp(\mathcal{D}_\theta^{\text{KL}})}. \quad (10)$$

This decomposition reveals that FPO implicitly corrects the PPO likelihood ratio with an *inverse KL gap*, accounting for the mismatch between flow policies across updates.

e) *Final Likelihood Ratio Expression*.: Using the surrogate loss yields:

$$r_\theta^{\text{FPO}} = \exp(\mathcal{L}_{\theta_{\text{old}}}^w(a_t) - \mathcal{L}_\theta^w(a_t)), \quad (11)$$

which plugs directly into the PPO clipped objective.

C. Flow Soft Actor-Critic (FSAC)

Flow-based generative models learn continuous-time velocity fields that transport noisy samples toward clean data [5, 6]. In FSAC, we reinterpret this denoising trajectory as a second, short-horizon *flow-MDP* nested inside the standard long-horizon environment MDP. Each flow step $k \in [0, 1]$ corresponds to a latent transition governed by the learned velocity field, while the final action at $k = 0$ interacts with the external environment.

This construction yields a *Dual-MDP* architecture in which:

- the **environment MDP** evolves according to (s_t, a_t, r_t, s_{t+1}) , and
- the **flow MDP** evolves according to $(a^{(k)}, v_\phi, r_{\text{flow}})$.

Our design is conceptually related to hierarchical multi-scale decision structures such as DPPO [13], which couple different temporal resolutions. However, FSAC differs by embedding the denoising dynamics directly into the actor itself, enabling online training of flow-based policies rather than treating flow guidance as a refinement layer.

The dual-MDP view offers two benefits: (1) the flow trajectory yields dense shaping signals at intermediate steps, and (2) the flow-based actor can be trained online, co-evolving with the environment dynamics, unlike prior offline-diffusion methods [2, 1].

Below, we summarize the transition dynamics and update rules that implement FSAC.

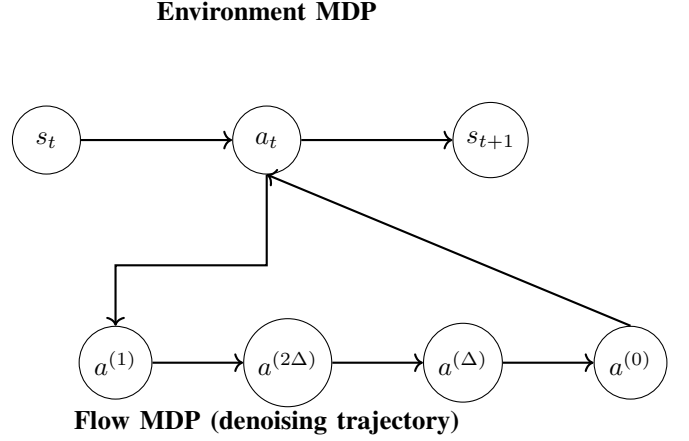


Fig. 1: FSAC treats each denoising step as a flow-MDP transition, with the final action passed to the environment.

1) *Flow Interpolation*: Each episode begins by sampling noise ε_t and defining a noisy trajectory via linear interpolation:

$$a_t^{(k)} = (1 - k) \varepsilon_t + k a_t^{(0)}, \quad k \in \{1, 1 - \Delta t, \dots, 0\}.$$

At flow step k , the policy observes:

$$\tilde{s}_t^{(k)} = \text{concat}(s_t, a_t^{(k)}).$$

2) *Velocity-Field Actor*: The flow-based actor outputs a velocity field:

$$v_t^{(k)} \sim \pi_\phi(v | \tilde{s}_t^{(k)}, k).$$

A reverse-flow denoising update is applied:

$$a_t^{(k-\Delta t)} = a_t^{(k)} + \Delta t v_t^{(k)}.$$

This updated action is treated as the action for Q-learning at flow step k .

3) *Flow Reward*: To guide the denoising dynamics with task objectives, FSAC introduces a shaped reward at each flow step:

$$v_t^{*,(k)} = a_t^{(0)} - \varepsilon_t,$$

$$r_{\text{flow}}(t, k) = \cos(v_t^{(k)}, v_t^{*,(k)}) r_t.$$

This reward structure encourages velocities that: (1) align with the ideal denoising direction, and (2) improve task performance.

Environment reward is applied only at the final step:

$$r_{\text{env}}(t, k) = \begin{cases} 0, & k < 1, \\ r_t, & k = 1. \end{cases}$$

Total reward:

$$\tilde{r}_t^{(k)} = r_{\text{flow}}(t, k) + r_{\text{env}}(t, k).$$

4) *Expanded SAC Transitions*: Each environment transition $(s_t, a_t^{(0)}, r_t, s_{t+1}, d_t)$ is expanded into multiple flow-MDP transitions:

$$\left(\tilde{s}_t^{(k)}, a_t^{(k)}, \tilde{r}_t^{(k)}, \tilde{s}_t^{(k-\Delta t)}, d_t^{(k)} \right),$$

with

$$d_t^{(k)} = \begin{cases} 0, & k < 1, \\ d_t, & k = 1. \end{cases}$$

These transitions are treated as standard SAC samples, providing dense supervision across the denoising trajectory.

5) *Algorithm Summary*: At each environment step:

- 1) Initialize noisy action $a_t^{(1)} = \varepsilon_t$.
- 2) For $k = 1, 1 - \Delta t, \dots, \Delta t$:
 - a) Form flow state $\tilde{s}_t^{(k)}$.
 - b) Sample velocity $v_t^{(k)}$.
 - c) Update action $a_t^{(k-\Delta t)} = a_t^{(k)} + \Delta t v_t^{(k)}$.
 - d) Compute shaped reward $\tilde{r}_t^{(k)}$.
 - e) Store the flow transition.
- 3) Execute final action $a_t^{(0)}$ in the environment.
- 4) Store environment reward and termination in the final transition.
- 5) Update critics and actor using SAC losses on the expanded dataset.

IV. EXPERIMENTS AND ANALYSIS

A. Experiment and hardware Setup

We evaluate our methods on standard Gymnasium continuous-control tasks. PPO and FPO are trained on a single RTX 5080 GPU using JAX, while SAC and FSAC are trained separately on a RTX 4070 laptop to distribute computational load. All experiments are logged and monitored using Weights & Biases. We evaluate the four RL methods—PPO, FPO, SAC, and FSAC—on Pendulum-v1, BipedalWalker-v3, and Reacher-v4, training each agent from scratch. The objective of these experiments is to analyze the fundamental capabilities and learning dynamics of flow-based RL methods before applying them to more challenging downstream fine-tuning tasks.

B. Baseline results

Table I summarizes the average episode return of the last 10 evaluation episodes across the three tasks. A key aspect of our setup is that PPO and FPO were trained for only 150 policy-update iterations, while SAC and FSAC were trained for substantially longer horizons. This reflects our focus on developing flow-based actor-critic methods, and therefore the PPO/FPO scores should be interpreted as short-horizon performance rather than asymptotic capability.

Under the limited training budget, both PPO and FPO reliably learn the simple Pendulum task (Fig. 2). For the more challenging long-horizon BipedalWalker environment (Fig. 3), FPO exhibits stronger early learning than PPO, highlighting its sample efficiency in the initial phase of training. However, in a subsequent longer run (500 steps, not shown in the table),

PPO ultimately learns both BipedalWalker and Reacher far more reliably than FPO. In contrast, FPO tends to stagnate due to insufficient exploration, as it assigns low probability to improved actions early on and therefore struggles to escape suboptimal behaviors. We also observed higher variance in FPO’s updates, which further contributes to instability and motivates the additional regularization and stabilization techniques used in the RL fine-tuning stage introduced in Section VI.

For the off-policy methods, SAC and FSAC were trained for substantially longer horizons, providing a more complete picture of asymptotic performance. On Pendulum-v1, FSAC achieves near-parity with SAC (-85 vs. -80), and crucially, both off-policy methods dramatically outperform the on-policy baselines (PPO: -550, FPO: -579). This demonstrates that the flow-based velocity policy within the dual-MDP formulation can learn effectively when the state-action space is low-dimensional and the task horizon is short.

However, as task complexity increases, a clear performance gap emerges. On Reacher-v4, FSAC (-14) still outperforms PPO (-56) and FPO (-52), but falls short of SAC (-4). The gap becomes most pronounced on BipedalWalker-v3: SAC converges to a return of 300, while FSAC plateaus at -13—a difference of over 300 reward units. This stark contrast suggests that the dual-MDP formulation introduces optimization challenges that compound with task complexity.

Figure 4 provides critical insight into this failure mode by comparing FSAC across different denoising horizons ($k_h = 5, 10, 20, 50$) against standard SAC. SAC (orange) rapidly converges to ~ 300 return by 200k steps and remains stable. In contrast, all FSAC variants fail to escape negative returns throughout training. Notably, the relationship between flow horizon and performance is non-monotonic: $k_h = 5$ (gray) achieves the best FSAC performance around -30, while $k_h = 50$ (green) exhibits the highest variance and worst stability, frequently dropping to -200. This suggests that longer denoising chains do not simply slow learning—they fundamentally destabilize the optimization landscape by diluting credit assignment across too many intermediate flow steps.

TABLE I: Average Episode Return (Last 10 Episodes)

Environment	FSAC	SAC	PPO	FPO
Pendulum-v1	-85	-80	-550	-579
Reacher-v4	-14	-4	-56	-52
BipedalWalker-v3	-13	300	380	95

C. Horizon Compounding in FSAC and Reward Design for the Flow-MDP

a) *Empirical symptom*: Figure 4 shows that standard SAC quickly reaches high returns on BipedalWalker-v3, while FSAC variants remain near negative returns and fail to converge. Moreover, increasing the denoising horizon (e.g., larger k_h) does not improve performance and can further destabilize training. This indicates that the difficulty is not simply insufficient exploration, but a deeper optimization issue.

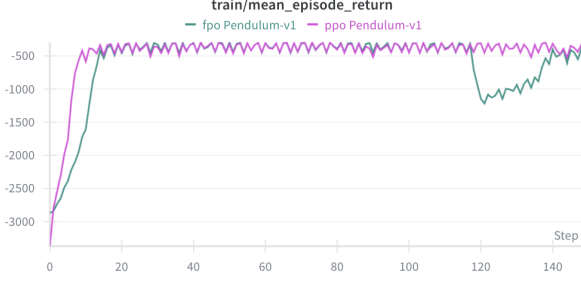


Fig. 2: Mean episode reward of Pendulum-v1 for PPO baseline and FPO at 150 policy update iterations.



Fig. 3: Mean episode reward of BipedalWalker-v3 for PPO baseline and FPO at 150 policy update iterations.

b) *Why FSAC suffers from horizon compounding.*: FSAC introduces a nested *flow-MDP* that generates an action through k_h denoising steps. While the environment MDP has horizon H_{env} , FSAC effectively expands the decision chain to

$$H_{\text{eff}} \approx H_{\text{env}} \cdot k_h, \quad (12)$$

because each environment action now depends on a sequence of intermediate denoising decisions. This enlarged effective horizon dilutes credit assignment: the environment reward r_t^{env} arrives after the entire denoising chain, but the learning

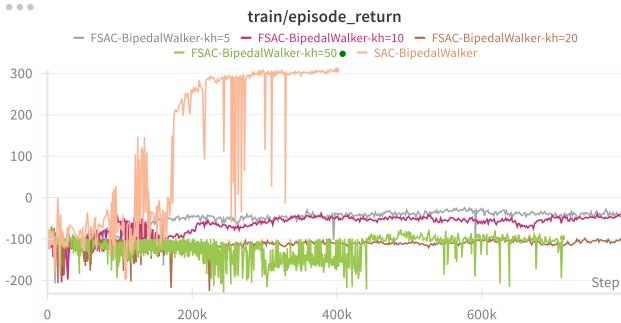


Fig. 4: Training episode return on BipedalWalker-v3. FSAC with different denoising horizons ($k_h = 5, 10, 20, 50$) is compared against standard SAC. While SAC converges to high returns, FSAC variants fail to stabilize, illustrating the horizon compounding effect introduced by multi-step flow denoising.

signal must be propagated through many flow steps. The resulting gradients become noisy and weak, particularly in high-dimensional continuous control where small action errors compound into falls and large negative returns (as in BipedalWalker-v3).

c) *Take Away* : The BipedalWalker results suggest that naive coupling of a multi-step denoising chain with off-policy value learning creates an *effective long-horizon RL problem*. Reward design for the flow-MDP must therefore be treated as a credit-assignment tool: dense, normalized, and temporally consistent shaping can improve stability, we need to more carefully introduce the environment reward into flow process, but aggressive horizon expansion (large k_h) can still overwhelm learning in high-dimensional continuous control, it is also a well-known problem in the off-policy RL.

V. CONCLUSION

This work introduced FPO and FSAC, two novel algorithms for Flow-Based RL. We derived the theoretical connection between Flow Matching loss and PPO updates and proposed a Dual-MDP architecture for off-policy learning. Our results highlight a fundamental trade-off: while flow policies offer superior expressivity and guided exploration, the nesting of flow steps within environment steps creates a *Horizon Compounding* effect that hinders credit assignment in complex tasks. Future work must address this by decoupling the flow horizon from the value propagation mechanism, perhaps via hierarchical temporal abstraction.

VI. FUTURE WORK

To further improve the flow-matching policy beyond training from scratch, we explored an online RL fine-tuning stage using FPO across two complex manipulation domains: (i) D4RL Kitchen-Partial, where human demonstrations perform individual sub-tasks but never the full four-task sequence, and (ii) AdroitHandPen-v1, where the demonstration data is constructed by training an imitation policy on a mixture of expert and human trajectories and then rolling out the policy. The resulting dataset, standardized through the Minari interface [20], captures a broad range of grasping and rotation behaviors in the Adroit environment.

In both settings, we first trained a conditional flow-matching (CFM) behavioral cloning policy that achieved reliable reproduction of behaviors present in the demonstrations. Starting from these strong BC initializations, we incorporated several stabilizing mechanisms—critic warmup, conservative KL constraints, BC anchoring, and step-size-limited actor updates—to prevent policy collapse during interaction. These techniques were applied uniformly across Kitchen and Pen.

Despite these modifications, FPO fine-tuning remained unstable in both environments. The sparse, discontinuous reward structures produced high-variance advantage estimates, leading to abrupt policy-ratio spikes and rapid degradation of the learned behavior. While the BC policies successfully executed individual Kitchen sub-tasks and produced near-correct pen rotations, online RL updates consistently disrupted this structure rather than improving task completion or reward.

These preliminary results indicate that flow-based RL remains a promising direction—particularly given its strong performance on simpler, dense-reward locomotion tasks such as Walker and HalfCheetah [12]—but that extending these methods to sparse-reward manipulation domains will require additional methodological advances. In particular, stable fine-tuning may depend on (i) shaped or curriculum rewards, (ii) offline-to-online advantage filtering or other forms of dataset regularization, or (iii) critic architectures explicitly designed for discontinuous manipulation rewards. Developing such techniques is an important avenue for future work. Full training curves, policy-ratio diagnostics, and rollout frames for both Kitchen and Pen are included in the Appendix (Figures 5 and 6).

VII. TEAM MEMBER CONTRIBUTION

Hua-Hsuan(Wilson) Liang(hl3811): Led the overall project design and execution. He was responsible for formulating the core research direction, including the adoption of Flow Policy Optimization (FPO) and the development of the novel Flow Soft Actor-Critic (FSAC) algorithm. His contributions include algorithmic design, theoretical formulation, and experimental validation. In addition, he implemented the codebase structure, and integrated all components of the learning pipeline, ensuring the project’s completeness and reproducibility.

Edward Zhang (cz2874): His primary contributions to the project were handling the FPO and PPO training pipelines. This involved adapting the original FPO and PPO implementations to our custom Gym environments, integrating them into our training framework, and performing the associated training, debugging, and hyperparameter tuning. He also implemented the full flow-based behavioral cloning and RL fine-tuning framework in D4RL. For FPO fine-tuning, he explored several stabilization techniques to address instability on the Kitchen and Pen tasks. In addition to running and managing the large-scale experiments, he prepared the resulting figures, analyses, and summaries that appear in the presentation and report.

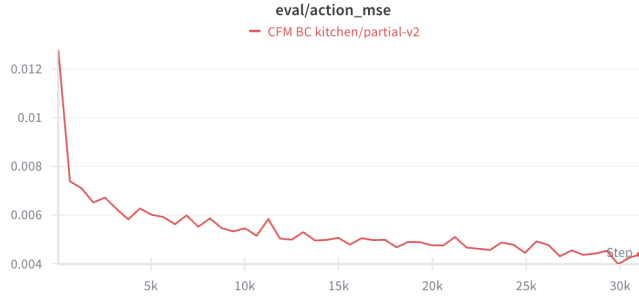
Jaisel Singh (js6897): Contributed primarily to the design, implementation, and evaluation of the Soft Actor-Critic (SAC) and conceptualizing limits of FSAC for the presentation. He implemented the SAC training pipelines. He tried helping with debugging fpo convergence issues initially and integrating the phc environment. He helped in writing and refining the technical sections of the report, including the off-policy method performance, FSAC methodology, horizon-compounding analysis, experimental discussion, and overall narrative coherence of the paper.

REFERENCES

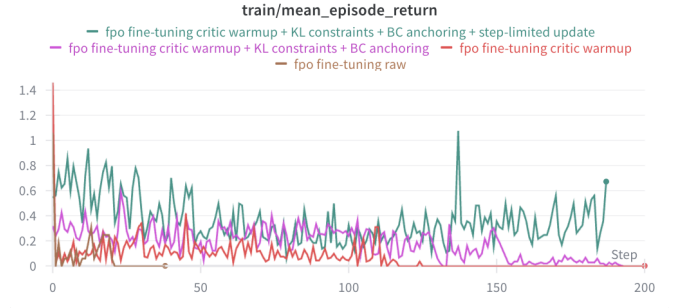
- [1] T. Pearce *et al.*, “Counter-strike: Diffusion policy for real-world robotics,” *arXiv preprint arXiv:2307.05442*, 2023.
- [2] M. Janner *et al.*, “Planning with diffusion for flexible behavior synthesis,” in *ICML*, 2022.
- [3] X. Zhai and C. Hao, “Vfp: Variational flow-matching policy for multi-modal robot manipulation,” *arXiv preprint arXiv:2508.01622*, 2025.
- [4] T. Lipman, J. Song, and S. Ermon, “Flow matching for generative modeling,” *arXiv preprint arXiv:2210.02747*, 2022.
- [5] Y. Lipman, S. Tishby, R. Chen *et al.*, “Flow matching for generative modeling,” in *ICLR*, 2023.
- [6] J. Liu, T. Q. Chen *et al.*, “Flow straight and fast: Learning to generate and manipulate images without diffusion,” *arXiv preprint arXiv:2209.00796*, 2022.
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” in *arXiv preprint arXiv:1707.06347*, 2017.
- [8] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” *arXiv preprint arXiv:1801.01290*, 2018.
- [9] S. Ross, G. J. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 627–635.
- [10] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *International Conference on Machine Learning (ICML)*, 2018, pp. 1582–1591.
- [11] J. Achiam, “Spinning up in deep reinforcement learning,” <https://spinningup.openai.com>, 2018.
- [12] D. McAllister, S. Ge, B. Yi, C. M. Kim, E. Weber, H. Choi, H. Feng, and A. Kanazawa, “Flow matching policy gradients,” in *arXiv preprint arXiv:2507.21053*, 2025.
- [13] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz, “Diffusion policy policy optimization,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.00588>
- [14] J. Ho and S. Ermon, “Generative adversarial imitation learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, 2016, pp. 4565–4573.
- [15] D. Pomerleau, “Alvin: An autonomous land vehicle in a neural network,” *NeurIPS*, 1989.
- [16] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” *NeurIPS*, 2020.
- [17] J. Fan, S. Shen, C. Cheng, Y. Chen, C. Liang, and G. Liu, “Online reward-weighted fine-tuning of flow matching with wasserstein regularization,” *arXiv preprint arXiv:2502.06061*, 2025.
- [18] M. Ward, X. Peng, and A. Grover, “Flow policies for reinforcement learning,” in *ICML*, 2024.
- [19] T. Haarnoja *et al.*, “Reinforcement learning with deep energy-based policies,” in *ICML*, 2017.
- [20] J. Francis, J. Terry, A. Kwiatkowski, B. Black *et al.*, “Minari: A standardized interface for offline reinforcement learning datasets,” <https://github.com/>

Farama-Foundation/minari, 2022, farama Foundation.

APPENDIX



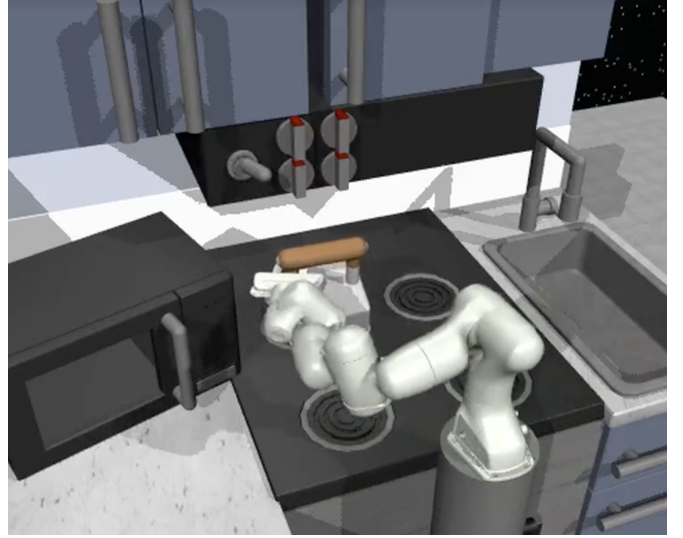
(a) Kitchen BC performance



(b) Kitchen RL reward during fine-tuning

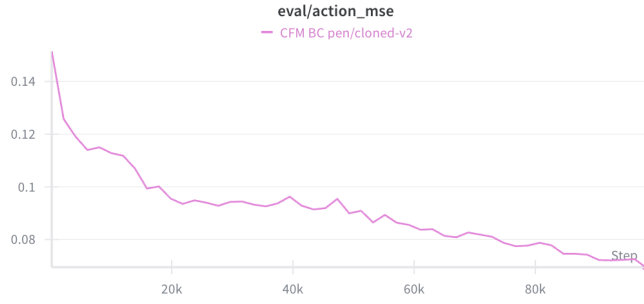


(c) Kitchen policy ratio (indicator of FPO stability)



(d) Example Kitchen rollout under the learned policy

Fig. 5: Appendix A: D4RL Kitchen-Partial. Flow-based behavioral cloning achieves reasonable performance on the individual sub-tasks. However, the large spikes in the maximum policy ratio indicate that the actor still receives overly aggressive updates, even after substantial stabilization efforts. The mean episode reward reflects some benefits from these measures, but they remain insufficient to enable stable policy improvement.



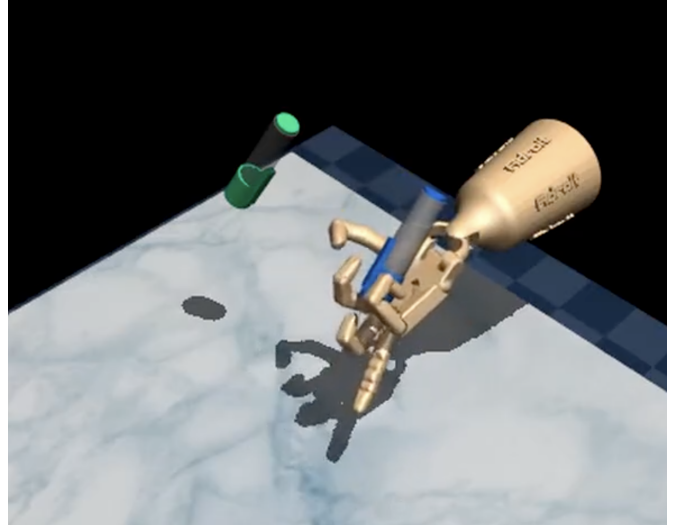
(a) Pen BC performance



(b) Pen RL reward during fine-tuning



(c) Pen policy ratio



(d) Example Adroit Pen rollout under the learned policy

Fig. 6: Appendix B: AdroitHandPen-v1. Top: flow-based BC reproduces pen orientations close to the demonstrations. Bottom: FPO fine-tuning destabilizes the policy, with reward collapse and large policy-ratio spikes. All stabilization interventions used for Kitchen-Partial were applied in this task, but were similarly insufficient.